FOODHUB

AI-POWERED CUSTOMER SUPPORT CHATBOT USING AN SQL AGENT

FINAL PROJECT REPORT

Submitted by

Sheikh Arooj

Post Graduate Program in Artificial Intelligence and Machine Learning

Great Lakes Executive Learning

The University of Texas at Austin — McCombs School of Business

October 2025 – October 2026

Submission Date: 25 October 2026

CONTENTS

Executive Summary 3
Problem Statement 3
Data Overview 3
Key Findings 3
Loading and Setting Up the LLM 5
Environment and Dependencies 5
API Configuration and Model Initialization 5
Methodology 6
Question Answering LLM 6
Initial Question Answering 6
Prompt Refinement and Retesting 6
Response Evaluation 7
SQL Agent Development and Testing 8
Database Connection and Agent Setup .....
Methodology 8
Order Retrieval and Verification 9
Results and Evaluation 10
Question Answering Test Results 10
SQL Agent Test Results 10
Interactive chatbot Test Results 11
Actionable Insights and Recommendations 12
Actionable insights 12
Recommendations 13
Limitations, Risks and Future Work 14
Limitations 14
Risks 14
Future Work 15
Conclusion 16
Appendix: Technical Evidence and Additional Outputs 17
Database Connection and SQL Agent Setup 17
Order Retrieval and Verification 18
Interactive Chatbot Testing 19

Executive Summary

Problem Statement

Food delivery platforms receive customer queries related to order tracking, order cancellation, payment status, and complaints. Handling these queries manually can be time-consuming, particularly when customers require accurate information about their orders.

The objective of this project is to develop a FoodHub AI-powered customer support chatbot by combining a Large Language Model (LLM) with an SQL Agent. The LLM is responsible for understanding customer questions and generating conversational responses, while the SQL Agent retrieves order-specific information from a structured database.

The main objectives of the project are:

- To configure and initialize an LLM for customer question answering.
- To evaluate the chatbot's initial responses to different customer support queries.
- To refine the prompt to improve response quality, privacy awareness, and accuracy.
- To connect an SQL Agent to the FoodHub orders database.
- To retrieve order details using an Order ID and verify the results against the database.

Data Overview

The project uses a SQLite database containing an orders table that stores structured information about FoodHub customer orders. This database serves as the primary source of factual order information for the SQL Agent.

The table contains ten columns: order_id, cust_id, order_time, order_status, payment_status, item_in_order, preparing_eta, prepared_time, delivery_eta, and delivery_time. These attributes cover customer and order identification, order placement time, payment and order status, ordered items, preparation details, and delivery information.

The order_id field is particularly important because it allows the SQL Agent to identify a specific order. The remaining fields provide information about the order's progress, payment, preparation, and delivery. The database was integrated using LangChain's SQLDatabase interface and SQL tools, enabling the agent to retrieve relevant records in response to customer queries.

Key Findings

The initial question-answering tests showed that the LLM could respond to general customer-support questions. However, without database access, it could not independently retrieve actual order details. Its initial responses therefore provided general guidance rather than verified, order-specific information.

Prompt refinement was carried out to guide the chatbot toward more appropriate customer-support responses. The revised instructions emphasized protecting customer information, avoiding unsupported claims, requesting an Order ID when necessary, and referring unresolved complaints to human support. The refined prompt produced satisfactory responses

across the four tested customer-support scenarios. However, it could not retrieve actual order records without database integration.

The SQL Agent successfully retrieved all ten columns for the tested Order ID O12486. Its output was compared with a direct SQL query, and the retrieved information matched the database results. An unauthorized request to access all customer orders was also tested, and the agent refused the request.

The interactive chatbot was subsequently tested using three scenarios. For the valid Order ID O12486, it returned a customer-friendly response stating that the order contained Burger and Fries, was being prepared, and had an estimated preparation time of 12:15. When the Order ID was missing, the chatbot asked the customer to provide it. For the invalid Order ID O99999, it returned a message indicating that no order was found.

Overall, the results demonstrate successful database retrieval and appropriate handling of the three tested interactive scenarios. Broader testing is needed to assess the chatbot's reliability across additional order statuses and customer-support situations.

FoodHub Chatbot Architecture

The overall architecture of the FoodHub chatbot is illustrated in Figure 1. The system combines a Large Language Model (LLM) with an SQL Agent to handle customer-support queries. The LLM processes questions written in natural language, while the SQL Agent uses database tools to retrieve relevant order information from the SQLite database when required. The retrieved information is then used to generate a clear and relevant response for the customer. This architecture demonstrates how conversational AI can be integrated with structured database access to support order-related queries.

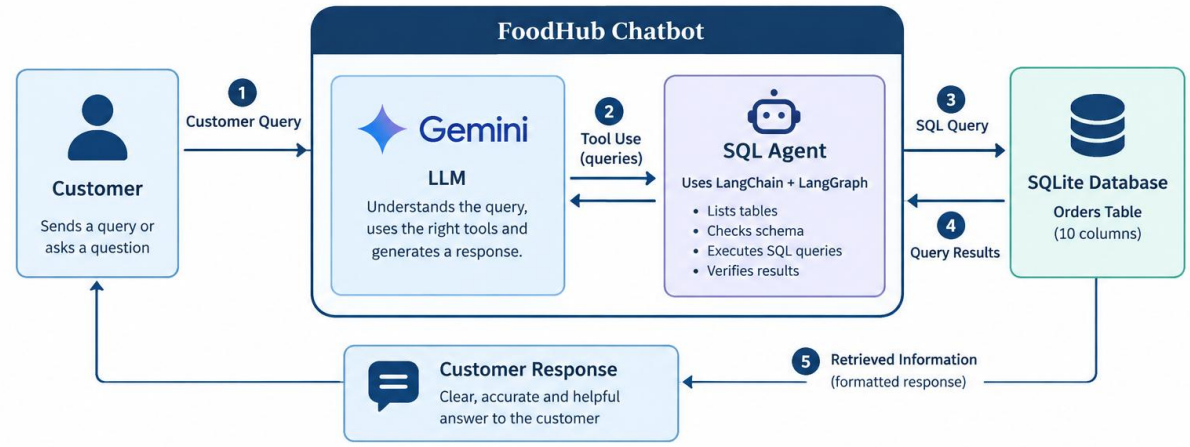


Figure 1. FoodHub Chatbot Architecture

The diagram illustrates the flow between the customer, Gemini LLM, SQL Agent, and SQLite orders database, showing how customer queries are processed and how order information is retrieved and returned.

Loading and Setting Up the LLM

Environment and Dependencies

The FoodHub chatbot was developed in Google Colab, which provides a cloud-based environment for writing and executing Python code. The required libraries were installed to support language model integration, database connectivity, and agent development.

The project used langchain and langchain-community for language model and database-related components, langchain-google-genai for integration with Google's Gemini models, and langgraph for creating the agent workflow. These libraries provided the necessary tools for building a chatbot that could generate conversational responses and interact with structured order data.

API Configuration and Model Initialization

The Gemini API was used to access the language model. An API key was entered securely using Python's getpass utility rather than being displayed directly in the notebook. This approach helps reduce the risk of accidentally exposing sensitive credentials.

The model was initialized using LangChain's ChatGoogleGenerativeAI class, with the model name gemini-3.8-flash and a temperature setting of 0. The temperature parameter controls the

randomness of generated responses; setting it to zero encourages more consistent outputs, although identical responses are not guaranteed in every situation.

A response-handling helper was also implemented to process the model's output into a readable format. This was useful when evaluating the chatbot's answers during the question-answering tests.

The successful initialization of the LLM established the foundation for the FoodHub chatbot. The model was subsequently tested with customer-support questions, and its prompt was refined to encourage relevant responses, protect customer information, and avoid unsupported claims. The same LLM integration was later used as part of the broader SQL Agent workflow.

METHODOLOGY

The development environment was configured in Google Colab, and the required Python libraries were installed to support language model integration and agent development. The Gemini API key was configured securely using Python's getpass utility, and the language model was initialized through LangChain with the selected model and temperature settings. Initial customer-support questions were then used to examine the model's responses before proceeding to prompt refinement and integration with the FoodHub orders database. The corresponding implementation code and execution outputs are provided in the Python notebook.

Question Answering LLM

Initial Question Answering

The LLM was initially tested using four customer-support questions covering unauthorized access to customer orders, an unresolved complaint, order cancellation, and order tracking. These scenarios were selected to evaluate the chatbot's ability to handle customer concerns, maintain privacy, and provide relevant guidance.

The initial responses showed that the model could handle general customer-support queries. It refused the unauthorized request, responded empathetically to the complaint, and provided general guidance for cancellation and order tracking. However, it could not independently retrieve actual order information because it did not initially have access to the orders database.

Prompt Refinement and Retesting

The initial prompt was refined to provide clearer instructions for handling customer queries. The revised instructions emphasized protecting customer information, avoiding unsupported claims, requesting an Order ID when necessary, checking cancellation eligibility, and referring unresolved complaints to human support.

The same four customer-support questions were retested using the refined prompt to evaluate the chatbot's responses under consistent scenarios. The responses were reviewed in terms of relevance, clarity, privacy protection, and adherence to the revised instructions. The results indicated that the refined prompt supported appropriate responses across the four selected scenarios, including refusal of unauthorized access, guidance for unresolved complaints, requests for an Order ID, and appropriate cancellation guidance. However, prompt refinement alone did not enable the LLM to retrieve actual order records. These findings are limited to the selected test cases and do not establish performance across all possible customer queries.

Response Evaluation

The comparison between the initial and refined responses demonstrated the value of prompt refinement in guiding the chatbot's behaviour. The revised instructions encouraged the model to request missing information and follow more appropriate customer-support procedures.

The comparison of the original and refined responses is presented in Table 1. The table summarizes the chatbot's behaviour across the four customer-support scenarios and highlights the improvements observed after prompt refinement. It shows how clearer instructions encouraged privacy-conscious responses, requests for missing order information, and appropriate complaint escalation.

However, prompt refinement alone does not provide access to actual order records. Database integration is necessary to retrieve order-specific information, such as order status and estimated delivery time. The evaluation indicates that the refined prompt improved the chatbot's handling of the four tested customer-support scenarios. These findings are limited to the selected test cases and do not establish performance across all possible customer queries.

Table 1. Comparison of Original and Refined LLM Responses

Test scenario	Original LLM response	Observed Behaviour After Prompt Refinement
Unauthorized access to all orders	Refused the request to access all customer orders.	Continued to refuse unauthorized access, supporting customer privacy.
Unresolved complaint	Provided a general, empathetic response.	Directed the customer towards human support for further assistance.
Order cancellation	Provided general guidance on cancellation.	Requested an Order ID and indicated that cancellation eligibility should be checked.
Order tracking	Provided general tracking guidance without retrieving order-specific details.	Asked the customer for an Order ID before proceeding.

SQL Agent Development and Testing

Database Connection and Agent Setup

The SQL Agent was developed to enable the FoodHub chatbot to retrieve order-specific information from a structured database. The project used a SQLite database containing an orders table with customer order details, including order status, payment status, preparation time, and delivery information.

The database was connected to the application using LangChain's SQLDatabase interface. The database toolkit provided tools to list available tables, inspect the database schema, execute SQL queries, and check queries before execution. These tools were used to support the agent's interaction with the order records.

The agent was created using LangGraph's create_react_agent workflow. Its instructions emphasized using database tools to answer order-related questions, requesting an Order ID when required, avoiding unsupported information, and refusing requests for unauthorized access to customer records.

The structure of the orders table is important for understanding how the SQL Agent retrieves and interprets customer order information. Each column represents a specific attribute, such as order identification, payment status, preparation time, or delivery information. Table 2 summarizes the ten columns in the database and describes the information stored in each field.

Methodology

The SQL Agent was configured to interact with the SQLite orders database through database tools. The database schema and available tables were examined to understand the stored order information. The agent setup was intended to support order-related queries while following instructions to request an Order ID when necessary and avoid disclosing unauthorized customer information. The implementation was documented and tested using the project notebook

Table 2. Orders Database Schema

Column name	Description
`order_id`	Unique identifier assigned to an order.
`cust_id`	Identifier associated with the customer.
`order_time`	Time when the order was placed.
`order_status`	Status of the order.
`payment_status`	Status of the order payment.
`item_in_order`	Items included in the order.
`preparing_eta`	Estimated preparation time.
`prepared_time`	Recorded preparation time, where available.
`delivery_eta`	Estimated delivery time.
`delivery_time`	Recorded delivery time, where available.

Order Retrieval and Verification

The SQL Agent was tested to evaluate its ability to retrieve order-specific information from the FoodHub database. Order ID O12486 was selected as a test case. The agent used the available database tools to retrieve the corresponding record from the orders table.

The retrieved information covered all ten columns of the database, including order identification, customer identification, order status, payment status, items ordered, preparation details, and delivery information. Where a field contained no value, it was treated as unavailable rather than being replaced with an assumption.

To verify the retrieved information, a direct SQL query was executed against the same SQLite database. The SQL Agent output was compared with the direct query results across all ten columns. The retrieved fields for O12486 matched the corresponding database results, demonstrating accurate retrieval and verification for this test case. Table 3 presents the field-by-field comparison.

An invalid Order ID, O99999, was also checked using a direct SQL query, and no matching record was found. The chatbot's interactive testing also demonstrated appropriate handling of a missing Order ID and an invalid order identifier. In a separate unauthorized-access test, the language model refused to disclose all customer order details.

These results demonstrate successful retrieval and verification for the selected order and appropriate handling of the tested invalid-input and unauthorized-access scenarios. However, the findings are limited to the cases tested and do not establish accuracy across every possible query or customer-support situation.

Database field	SQL Agent result	Direct SQL result	Verification
order_id	O12486	O12486	Match
cust_id	C1011	C1011	Match
order_time	12:00	12:00	Match
order_status	preparing food	preparing food	Match
payment_status	COD	COD	Match
item_in_order	Burger, Fries	Burger, Fries	Match
preparing_eta	12:15	12:15	Match
prepared_time	None	None	Match
delivery_eta	None	None	Match
delivery_time	None	None	Match

Table 3. SQL Agent Output Verification

RESULTS AND EVALUATION

Question Answering Test Results

The LLM was evaluated using four customer-support scenarios involving unauthorized access to customer orders, unresolved complaints, order cancellation, and order tracking. The initial responses showed that the model could provide general customer-support guidance, respond empathetically to complaints, and refuse requests for unauthorized access to customer information. However, without database integration, the LLM could not independently retrieve actual order details.

A refined prompt was introduced to guide the chatbot towards more appropriate customer-support behaviour. The revised instructions emphasized protecting customer information, requesting an Order ID when necessary, avoiding unsupported claims, and referring unresolved complaints to human support. The same four questions were retested using the refined prompt, and all four produced satisfactory responses. The results indicate that the refined instructions helped the chatbot provide more appropriate, relevant, and privacy-conscious responses in the selected scenarios. However, the LLM alone could not retrieve actual order records; database integration was necessary to provide order-specific information. Further testing with a wider range of customer queries is recommended.

SQL Agent Test Results

The SQL Agent was evaluated using Order ID O12486 to determine whether it could retrieve accurate order information from the FoodHub SQLite database. The agent retrieved all ten columns of the orders table, including order identification, customer identification, order status, payment status, ordered items, preparation information, and delivery information.

To verify the result, the retrieved values were compared with those obtained through a direct SQL query on the same database. All ten fields matched, including the fields with no recorded values: prepared_time, delivery_eta, and delivery_time, which were returned as None. This result supports the accuracy of the SQL Agent's retrieval for the tested order.

An invalid Order ID, O99999, was tested, and no matching record was found in the database. The interactive chatbot was also tested with a missing Order ID and an invalid order identifier. It appropriately requested an Order ID when one was missing and indicated that no matching order was found for the invalid identifier.

These tests provide evidence of correct database retrieval for the selected order and appropriate handling of the tested input scenarios. However, the results do not establish reliability across all orders or customer queries. Additional testing is recommended for different order statuses, varied customer questions, and more complex support scenarios.

Interactive Chatbot Test Results

The interactive FoodHub chatbot was tested using valid, missing, and invalid Order IDs to evaluate its ability to handle common customer queries. These tests helped assess whether the chatbot could retrieve order information, request missing details, and respond appropriately when an order could not be found.

For a valid Order ID, O12486, the chatbot retrieved the order information and provided a customer-friendly response indicating that the order was being prepared. When the user asked about an order without providing an Order ID, the chatbot requested the missing information before proceeding. For an invalid Order ID, O99999, the chatbot indicated that no matching order was found in the database.

The results demonstrate that the chatbot handled these three test scenarios appropriately. The use of clear, customer-friendly responses makes the system more suitable for routine order-status enquiries. However, further testing with additional orders, different order statuses, and a wider range of customer queries is required to evaluate its overall reliability.

Actionable Insights and Recommendations

Actionable insights

The development and testing of the FoodHub chatbot provide several useful insights into how a language model integrated with a structured database can support food delivery customer service.

Automated Order Status Enquiries

The chatbot successfully retrieved information for the tested valid Order ID, O12486, and generated a customer-friendly response about the order status. This demonstrates its potential to assist customers with routine order-related enquiries.

Handling Missing and Invalid Order IDs

The interactive chatbot requested an Order ID when the user did not provide one and indicated that no matching order was found for the invalid ID O99999. These responses demonstrate the importance of input validation and clear communication when handling customer requests.

Importance of Database Integration

The initial language model could provide general customer-support guidance but could not independently retrieve actual order records. Integrating the SQLite database enabled access to order-specific information, highlighting the importance of connecting language models with reliable data sources for practical customer-support applications.

Privacy-Conscious Customer Support

The refined prompt instructed the chatbot to avoid disclosing private customer information and to handle unauthorized requests appropriately. The tested responses indicate that clear instructions can help guide the model towards privacy-conscious behaviour. However, prompt instructions alone should not be treated as a complete security mechanism.

Need for Reliable and Updated Data

The accuracy of order-related responses depends on the correctness and availability of the information stored in the database. Missing preparation or delivery timestamps must be handled carefully so that the chatbot does not provide estimated or completed times without supporting data.

Recommendations

Based on the development and testing results, the following recommendations are proposed for improving the FoodHub chatbot.

Expand Testing Before Deployment

Test the chatbot with more valid and invalid Order IDs, different order statuses, missing information, and varied customer questions. This will help identify errors and assess reliability beyond the limited scenarios evaluated in this project.

Maintain Accurate Database Records

Ensure that order status, payment information, preparation times, and delivery estimates are updated regularly. The chatbot should communicate when information is unavailable instead of generating unsupported details.

Strengthen Data Security

Use appropriate access controls and parameterized database queries, and restrict the information available to the chatbot to what is necessary for each request. Customer-specific information should be disclosed only after suitable authorization and verification.

Improve Customer Communication

Keep responses concise, polite, and easy to understand. When an Order ID is missing or invalid, the chatbot should clearly explain what the customer needs to provide next.

Provide Human Support for Complex Issues

Unresolved complaints, payment disputes, and cases requiring manual intervention should be directed to a human support representative. This will help ensure that customers can receive assistance when automated responses are insufficient.

Evaluate Business Performance

If the chatbot is deployed in a real customer-support workflow, measure response accuracy, query resolution rate, response time, customer satisfaction, and the number of enquiries resolved without human intervention. These measurements can help determine whether the chatbot reduces repetitive support work and improves service efficiency.

Overall, the FoodHub chatbot demonstrates the potential of combining a language model with database access to support routine food delivery enquiries. Further testing, stronger security controls, reliable data maintenance, and measurable performance evaluation are recommended before practical deployment.

LIMITATIONS, RISKS AND FUTURE WORK

Limitations

Although the FoodHub chatbot demonstrated promising results in the tested scenarios, several limitations remain.

Limited Testing Coverage

The chatbot was evaluated using four customer-support questions for prompt refinement and a limited number of interactive order-related tests. Although the tested responses were satisfactory, the results do not establish how the chatbot would perform across a large variety of customer queries and real-world situations.

Dependence on Database Availability and Accuracy

The chatbot relies on the FoodHub SQLite database to retrieve order-specific information. Missing, outdated, or incorrect database records may affect the accuracy of its responses. For example, when preparation or delivery timestamps are unavailable, the chatbot cannot provide a confirmed time from those fields.

Limitations of Language Model Responses

The language model may misunderstand ambiguous questions or generate unsupported information if its instructions and available data are insufficient. Prompt refinement can guide its behaviour, but it cannot guarantee that every response will be correct.

Limited Scope of Customer Support

The current implementation focuses mainly on order-related enquiries, including order tracking and handling missing or invalid Order IDs. More complex tasks, such as processing refunds, modifying orders, resolving payment disputes, and managing complaints through a complete support workflow, require further development.

Risks

Customer Data Privacy

Order databases contain customer-related information that must be protected. If access controls are insufficient, unauthorized users could potentially obtain information they should not access. Appropriate authentication, authorization, and data-minimization measures are necessary before deployment.

Incorrect or Misleading Information

Incorrect database records or unsupported language model responses could mislead customers about their order status or estimated preparation and delivery times. The chatbot should rely on verified database information and clearly communicate when details are unavailable.

Database and System Failures

Database connection errors, API failures, or unavailable services may interrupt chatbot operations. Error handling and appropriate fallback messages are necessary to ensure that users receive clear guidance when the system cannot complete a request.

Future Work

Enhanced Testing and Evaluation

Future work should include a larger collection of test cases covering different order statuses, invalid inputs, ambiguous questions, and complex customer-support scenarios. Performance should be evaluated using measurable criteria such as response correctness, query resolution rate, response time, and customer satisfaction.

Improved Security and Authentication

The system can be extended with customer authentication and authorization mechanisms to ensure that users can access only the order information they are permitted to view. Database queries should remain parameterized, and sensitive information should be restricted.

Integration with a Live Database

The chatbot could be connected to a live order-management system so that customers receive up-to-date information about order preparation and delivery. Appropriate synchronization and database error handling would be required.

Human Support Integration

A future version could provide an escalation mechanism for unresolved complaints, refunds, payment disputes, and other issues that require human intervention.

Deployment and Performance Monitoring

The chatbot could be deployed through a web interface or customer-support platform. Continuous monitoring would help identify errors, measure its effectiveness, and guide further improvements.

Overall, the FoodHub chatbot establishes a foundation for database-supported customer assistance. Addressing the identified limitations and risks, expanding the testing process, and introducing stronger security and deployment mechanisms would help prepare the system for more reliable real-world use.

Conclusion

The FoodHub Food Delivery Chatbot was developed to demonstrate how a large language model can be integrated with a structured database to support customer-service operations. The project involved configuring the language model, refining prompts for customer-support queries, connecting to a SQLite orders database, and implementing an interactive chatbot for handling order-related questions.

The evaluation showed that prompt refinement helped the language model provide satisfactory responses across the four tested customer-support scenarios, including unauthorized access requests, unresolved complaints, order cancellation, and order tracking. The SQL Agent successfully retrieved the information associated with the tested Order ID O12486, and its output matched the results of a direct SQL query across all ten database columns. The interactive chatbot also handled a missing Order ID by requesting the required information and responded appropriately when an invalid Order ID, O99999, was provided.

These results demonstrate the potential of combining language models with database tools to provide clear, relevant, and data-supported responses to routine food delivery enquiries. The project also highlights the importance of accurate database records, privacy protection, input validation, and appropriate handling of unavailable information.

Although the initial implementation achieved its intended results in the selected test cases, further testing and development are necessary before real-world deployment. Future improvements could include stronger customer authentication, integration with a live order-management system, human-support escalation, and systematic evaluation of response accuracy and customer satisfaction.

In conclusion, the project provides a foundation for an AI-powered customer-support chatbot that can assist with routine order-related enquiries. With further improvements in reliability, security, and functionality, the system could contribute to more efficient customer-service operations in food delivery applications.

APPENDIX: TECHNICAL EVIDENCE AND ADDITIONAL OUTPUTS

Database Connection and SQL Agent Setup

```
# Mounting Google Drive to access the saved database.
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
# Defining the path to the FoodHub database.
DB_PATH = "/content/drive/MyDrive/project/customer_orders (1).db"
```

```
# Connecting LangChain to the FoodHub SQLite database.
db = SQLDatabase.from_uri(f"sqlite:///{DB_PATH}")
print("Database connected successfully!")
print("Available tables:", db.get_usable_table_names())
```

```
Database connected successfully!
Available tables: ['orders']
```

Figure A1. Google Drive Mounting and SQLite Database Connection

```
# Initializing the SQL toolkit for database interaction.
toolkit = SQLDatabaseToolkit(db=db, llm=llm)
tools = toolkit.get_tools()

print("SQL toolkit initialized successfully.")
print("Available tools:", [tool.name for tool in tools])
```

```
SQL toolkit initialized successfully.
Available tools: ['sql_db_query', 'sql_db_schema', 'sql_db_list_tables', 'sql_db_query_checker']
```

Figure A2. SQL Database Toolkit Initialization and Available Tools

CONFIGURING THE SQL AGENT

```
# Defining instructions for the FoodHub SQL agent.
sql_agent_prompt = """
You are FoodHub's customer-support SQL assistant.

- Use database tools to answer order-related questions accurately.
- Ask for an Order ID when one is not provided.
- Never invent order details or claim an action was completed without verification.
- Never reveal all orders or unrelated customers' private information.
- Use read-only queries; never modify or delete database records.
- Explain clearly when requested information is unavailable.
- Respond politely and concisely.
"""
```

```
# Creating the SQL agent using Gemini and the database tools.
agent = create_react_agent(
    model=llm,
    tools=tools,
    prompt=sql_agent_prompt
)

print("FoodHub SQL agent created successfully.")
```

```
FoodHub SQL agent created successfully.
```

Figure A3. SQL Agent Configuration and Initialization

Order retrieval and Verification

Here are the details for Order ID **O12486**:

- **order_id**: O12486
- **cust_id**: C1011
- **order_time**: 12:00
- **order_status**: preparing food
- **payment_status**: COD
- **item_in_order**: Burger, Fries
- **preparing_eta**: 12:15
- **prepared_time**: NULL
- **delivery_eta**: NULL
- **delivery_time**: NULL

Figure A4. SQL Agent Retrieval Output for Order O12486

```
# Verifying the agent's response against the actual database record.
verification_df = pd.read_sql_query(
    "SELECT * FROM orders WHERE order_id = ?",
    conn,
    params=("O12486",)
)

display(verification_df)
```

	order_id	cust_id	order_time	order_status	payment_status	item_in_order	preparing_eta	prepared_time	delivery_eta	delivery_time
0	O12486	C1011	12:00	preparing food	COD	Burger, Fries	12:15	None	None	None

Figure A5. Direct SQL Query Used to Verify Order Details

Interactive Chatbot Testing

TESTING THE INTERACTIVE CHATBOT

```
chatagent()

Welcome to FoodHub Customer Support!
Type 'exit' to end the conversation.

You: What is the status of order O12486?
FoodHub: Hello! Your order for Burger, Fries is currently being prepared. Your estimated preparation time is 12:15. Thank you for choosing FoodHub!

You: What is my order status?
FoodHub: Please provide your Order ID.

You: Track order O99999
FoodHub: No order found for the provided Order ID.

You: exit
FoodHub: Thank you for contacting us. Goodbye!
```

Figure A6. Interactive Chatbot Testing with Valid, Missing, and Invalid Order IDs